

From the Command Line to the Browser: Design and Implementation of an Online XATS2JS Compilation Environment

Abstract

ATS is a functional programming language that emphasizes type safety and program verification. Its dependent and linear type systems make it possible to express and verify sophisticated program properties during compilation. However, the ATS toolchain normally depends on a locally configured command-line environment, which can create a barrier for first-time users. Based on the open-source XATS2JS project published by the ATS community on GitHub, I developed a browser-based environment that compiles ATS/Xanadu source code into JavaScript, together with a visual web interface. Users can enter ATS code directly in a web page, inspect the generated JavaScript, view raw compiler output and diagnostic messages, and browse the Prelude files required by the compiler. This article documents the motivation, system design, implementation details, development challenges, and current limitations of the project.

Keywords: ATS; Xanadu; XATS2JS; compiler; JavaScript; browser

1. Background and Objectives

ATS offers a highly expressive type system and strong support for program verification. Nevertheless, installing the toolchain, configuring environment variables, and learning the command-line compilation workflow can make quick experimentation difficult for new users. A conventional online compiler generally sends source code to a server and invokes a compiler on the back end. Such an architecture requires a maintained server environment and introduces additional deployment and communication costs. This project instead attempts to move the XATS2JS compilation process into the browser so that users can compile small ATS programs without installing the ATS toolchain locally.

The main objectives were to run the originally Node.js-dependent XATS2JS compiler inside a browser, construct a virtual file system accessible to the compiler, load the ATS Prelude library, capture standard output and diagnostic information, and provide source input, result display, and Prelude browsing through a React website.

2. System Architecture and Compilation Process

The system consists of four main parts: the compiler, a browser compatibility layer, a file-resource layer, and the user interface. The compiler is the JavaScript artifact generated by the ATS community project. The compatibility layer emulates the subset of Node.js facilities required by that artifact. The file-resource layer stores user input and Prelude files, while the user interface coordinates compilation and presents its results.

After the user selects “Compile,” the application first registers the submitted source text as ``input.dats`` in an in-memory file map. It then constructs a command-line-like ``process.argv``, supplies the input filename to the compiler, and invokes the entry function exposed by the generated XATS2JS code. Output produced during compilation is written to simulated ``stdout`` and ``stderr`` objects. When compilation finishes, the web interface extracts the generated JavaScript and displays it together with any errors or diagnostic information.

The process can be summarized as follows:

```
Enter ATS source code
  → Register it in the virtual file system
  → Construct compiler arguments
  → Invoke XATS2JS
  → Capture output and exceptions
  → Process and display the generated JavaScript
```

To improve readability, I added a lightweight post-processing stage. It removes selected comments, collapses excessive blank lines, and simplifies certain cases in which a temporary variable is immediately returned. The interface still preserves the raw standard output, allowing users to compare the original and processed results and making the system easier to debug.

3. Adapting the Runtime to the Browser

The central challenge of the project was not the visual interface but the migration of the compiler's runtime environment. Although XATS2JS had already been compiled into JavaScript, the generated program still assumed that it would execute under Node.js. It attempted to access ``process``, ``require``, the file system, and standard output, none of which are natively available in an ordinary browser page.

To address this problem, I implemented a browser compatibility layer. This layer creates a runtime object on ``globalThis`` containing a file map, simulated standard-output streams, and methods for registering and reading files. I also provided a restricted implementation of ``require("fs")``, allowing the compiler to use ``readFileSync`` and ``accessSync`` against the in-memory file map. XATS paths may contain prefixes such as ``$(XATSHOME)`` or ``prelude/``, so the lookup mechanism normalizes these forms and attempts several candidate paths for the same resource.

The number of Prelude files made manual imports impractical. I therefore wrote a file-map generation utility that traverses the Prelude directory and converts each path and file body into JavaScript data that can be imported by the front end. During website initialization, these resources are registered in the virtual file system, providing the compiler with a relatively complete foundational environment.

4. Website Design

The website is built with React and Vite and uses routing to separate the compiler page from the Prelude Browser. The compiler page places the source editor and output view side by side: users enter ATS/Xanadu code on the left, while the generated JavaScript, raw output, and diagnostic messages appear on the right. This arrangement makes the relationship between the source program and its compilation result easier to examine.

The Prelude Browser dynamically constructs a directory tree from the stored file paths. Users can expand directories, select a file, and inspect its contents in the page. In addition to exposing the resources embedded in the application, this feature supports the study of foundational ATS definitions and helps diagnose missing-file problems during compilation.

Because compilation takes place in the user's browser, source code does not need to be uploaded to a server. This simplifies the deployment architecture and allows the project to operate as a largely self-contained static website.

5. Development Experience and Limitations

This project showed me that compiling a compiler into JavaScript does not automatically make it browser-compatible. Most of the migration work occurs at the runtime boundary: determining where files are read from, how command-line arguments are passed, where output is written, and how host-environment dependencies should be replaced. Rather than reimplementing the compilation logic, I chose to construct a small and explicit compatibility layer around the existing compiler. This became one of the project's most important design decisions.

The current system still has several limitations. Its virtual file system primarily supports the embedded Prelude and a single user input file. Output processing is based on textual rules and is not a substitute for a formal optimization stage. The editor does not yet provide syntax highlighting, completion, or source-level error locations. Future work could add multi-file projects, user file imports, online execution of generated programs, and more structured compiler diagnostics.

6. Conclusion

Building on the existing work of the ATS community, this project migrates XATS2JS from a command-line execution model to the browser and provides a web tool that combines source input, compilation-result display, error diagnostics, and Prelude browsing. It is not only a front-end development exercise but also an engineering study of compiler-runtime migration, interface compatibility, and resource organization. Through this work, I gained a more concrete understanding of the ATS toolchain and of the differences between browser and Node.js execution environments.